project3/model.py

```python
## Building and training a bigram language model
from functools import partial
import math

import torch
import torch.nn as nn
from einops import einsum, reduce, rearrange

from config import BigramConfig, MiniGPTConfig

class BigramLanguageModel(nn.Module):
    """
    Class definition for a simple bigram language model.
    """

    def __init__(self, config):
        """
        Initialize the bigram language model with the given configuration.

        Args:
        config : BigramConfig (Defined in config.py)
            Configuration object containing the model parameters.

        The model should have the following layers:
        1. An embedding layer that maps tokens to embeddings. (self.embeddings)
           You can use the Embedding layer from PyTorch.
        2. A linear layer that maps embeddings to logits. (self.linear) **set bias to True**
        3. A dropout layer. (self.dropout)

        NOTE : PLEASE KEEP OF EACH LAYER AS PROVIDED BELOW TO FACILITATE TESTING.
        """

        super().__init__()
        # ========= TODO : START ========= #

        self.embeddings = nn.Embedding(config.vocab_size, config.embed_dim)
        self.linear = nn.Linear(config.embed_dim, config.vocab_size, bias=True)
        self.dropout = nn.Dropout(config.dropout)

        # ========= TODO : END ========= #

        self.apply(self._init_weights)

    def forward(self, x):
        """
        Forward pass of the bigram language model.

        Args:
        x : torch.Tensor
            A tensor of shape (batch_size, 1) containing the input tokens.

```

```
        Output:
        torch.Tensor
            A tensor of shape (batch_size, vocab_size) containing the logits.
        """

        # ========= TODO : START ========= #

        emb = self.embeddings(x)
        emb = self.dropout(emb)
        logits = self.linear(emb)
        return logits

        # ========= TODO : END ========= #

    def _init_weights(self, module):
        """
        Weight initialization for better convergence.

        NOTE : You do not need to modify this function.
        """

        if isinstance(module, nn.Linear):
            torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)
            if module.bias is not None:
                torch.nn.init.zeros_(module.bias)
        elif isinstance(module, nn.Embedding):
            torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)

    def generate(self, context, max_new_tokens=100):
        """
        Use the model to generate new tokens given a context.
        We will perform multinomial sampling which is very similar to greedy sampling,
        but instead of taking the token with the highest probability, we sample the next token from a multinomial distribution.

        Remember in Bigram Language Model, we are only using the last token to predict the next token.
        You should sample the next token x_t from the distribution p(x_t | x_{t-1}).

        Args:
        context : torch.Tensor
            The context is a sequence of tokens, where each token is an index in the vocabulary.
        max_new_tokens : int
            The maximum number of new tokens to generate.

        Output:
        torch.Tensor
            A tensor containing the generated tokens.
        """

        ### ========= TODO : START ========= ###
```

```
        self.eval()
        device = context.device
        generated = context.tolist()
        with torch.no_grad():
            for _ in range(max_new_tokens):
                last_token = torch.tensor([[generated[-1]]], device=device)
                logits = self.forward(last_token)
                logits = logits[0, -1, :]
                probs = torch.softmax(logits, dim=-1)
                next_token = torch.multinomial(probs, num_samples=1).item()
                generated.append(next_token)
        return torch.tensor(generated, device=device)

        ### ========= TODO : END ========= ###


class SingleHeadAttention(nn.Module):
    """
    Class definition for Single Head Causal Self Attention Layer.

    As in Attention is All You Need (https://arxiv.org/pdf/1706.03762)

    """

    def __init__(
        self,
        input_dim,
        output_key_query_dim=None,
        output_value_dim=None,
        dropout=0.1,
        max_len=512,
    ):
        """
        Initialize the Single Head Attention Layer.

        The model should have the following layers:
        1. A linear layer for key. (self.key) **set bias to False**
        2. A linear layer for query. (self.query) **set bias to False**
        3. A linear layer for value. (self.value) # **set bias to False**
        4. A dropout layer. (self.dropout)
        5. A causal mask. (self.causal_mask) This should be registered as a
buffer.
            - You can use the torch.tril function to create a lower triangular
matrix.
            - In the skeleton we use register_buffer to register the causal mask as
a buffer.
              This is typically used to register a buffer that should not to be
considered a model parameter.

        NOTE : Please make sure that the causal mask is upper triangular and not
lower triangular (this helps in setting up the test cases, )
        NOTE : PLEASE KEEP OF EACH LAYER AS PROVIDED BELOW TO FACILITATE TESTING.
        """
```

```
        super().__init__()

        self.input_dim = input_dim
        if output_key_query_dim:
            self.output_key_query_dim = output_key_query_dim
        else:
            self.output_key_query_dim = input_dim

        if output_value_dim:
            self.output_value_dim = output_value_dim
        else:
            self.output_value_dim = input_dim

        causal_mask = None  # You have to implement this, currently just a placeholder

        # ========= TODO : START ========= #

        self.key = nn.Linear(input_dim, self.output_key_query_dim, bias=False)
        self.query = nn.Linear(input_dim, self.output_key_query_dim, bias=False)
        self.value = nn.Linear(input_dim, self.output_value_dim, bias=False)
        self.dropout = nn.Dropout(dropout)
        causal_mask = torch.triu(
            torch.ones(max_len, max_len, dtype=torch.bool), diagonal=1
        )

        # ========= TODO : END ========= #

        self.register_buffer(
            "causal_mask", causal_mask
        )  # Registering as buffer to avoid backpropagation

    def forward(self, x):
        """
        Forward pass of the Single Head Attention Layer.

        Args:
        x : torch.Tensor
            A tensor of shape (batch_size, num_tokens, token_dim) containing the input tokens.

        Output:
        torch.Tensor
            A tensor of shape (batch_size, num_tokens, output_value_dim) containing the output tokens.

        Hint:
        - You need to 'trim' the causal mask to the size of the input tensor.
        """

        # ========= TODO : START ========= #

        B, T, _ = x.shape
        k = self.key(x)
```

```
        q = self.query(x)
        v = self.value(x)

        scores = q @ k.transpose(-2, -1) / math.sqrt(self.output_key_query_dim)
        mask = self.causal_mask[:T, :T]
        scores = scores.masked_fill(mask, float("-inf"))

        attn = torch.softmax(scores, dim=-1)
        attn = self.dropout(attn)
        out = attn @ v
        return out

        # ========= TODO : END ========= #


class MultiHeadAttention(nn.Module):
    """
    Class definition for Multi Head Attention Layer.

    As in Attention is All You Need (https://arxiv.org/pdf/1706.03762)
    """

    def __init__(self, input_dim, num_heads, dropout=0.1) -> None:
        """
        Initialize the Multi Head Attention Layer.

        The model should have the following layers:
        1. Multiple SingleHeadAttention layers. (self.head_{i}) Use setattr to dynamically set the layers.
        2. A linear layer for output. (self.out) **set bias to True**
        3. A dropout layer. (self.dropout) Apply dropout to the output of the out layer.

        NOTE : PLEASE KEEP OF EACH LAYER AS PROVIDED BELOW TO FACILITATE TESTING.
        """
        super().__init__()

        self.input_dim = input_dim
        self.num_heads = num_heads

        # ========= TODO : START ========= #

        head_dim = input_dim // num_heads
        for i in range(num_heads):
            setattr(
                self,
                f"head_{i}",
                SingleHeadAttention(
                    input_dim,
                    output_key_query_dim=head_dim,
                    output_value_dim=head_dim,
                    dropout=dropout,
                ),
            )
```

```
        self.out = nn.Linear(input_dim, input_dim, bias=True)
        self.dropout = nn.Dropout(dropout)

        # ========= TODO : END ========= #

    def forward(self, x):
        """
        Forward pass of the Multi Head Attention Layer.

        Args:
        x : torch.Tensor
            A tensor of shape (batch_size, num_tokens, token_dim) containing the input tokens.

        Output:
        torch.Tensor
            A tensor of shape (batch_size, num_tokens, token_dim) containing the output tokens.
        """

        # ========= TODO : START ========= #

        head_outputs = []
        for i in range(self.num_heads):
            head = getattr(self, f"head_{i}")
            head_outputs.append(head(x))
        concat = torch.cat(head_outputs, dim=-1)
        out = self.out(concat)
        out = self.dropout(out)
        return out

        # ========= TODO : END ========= #


class FeedForwardLayer(nn.Module):
    """
    Class definition for Feed Forward Layer.
    """

    def __init__(self, input_dim, feedforward_dim=None, dropout=0.1):
        """
        Initialize the Feed Forward Layer.

        The model should have the following layers:
        1. A linear layer for the feedforward network. (self.fc1) **set bias to True**
        2. A GELU activation function. (self.activation)
        3. A linear layer for the feedforward network. (self.fc2) ** set bias to True**
        4. A dropout layer. (self.dropout)

        NOTE : PLEASE KEEP OF EACH LAYER AS PROVIDED BELOW TO FACILITATE TESTING.
        """
        super().__init__()
```

```
303
304         if feedforward_dim is None:
305             feedforward_dim = input_dim * 4
306
307         # ========= TODO : START ========= #
308
309         self.fc1 = nn.Linear(input_dim, feedforward_dim, bias=True)
310         self.activation = nn.GELU()
311         self.fc2 = nn.Linear(feedforward_dim, input_dim, bias=True)
312         self.dropout = nn.Dropout(dropout)
313
314         # ========= TODO : END ========= #
315
316     def forward(self, x):
317         """
318         Forward pass of the Feed Forward Layer.
319
320         Args:
321         x : torch.Tensor
322             A tensor of shape (batch_size, num_tokens, token_dim) containing the
    input tokens.
323
324         Output:
325         torch.Tensor
326             A tensor of shape (batch_size, num_tokens, token_dim) containing the
    output tokens.
327         """
328
329         ### ========= TODO : START ========= ###
330
331         h = self.fc1(x)
332         h = self.activation(h)
333         h = self.fc2(h)
334         h = self.dropout(h)
335         return h
336
337         ### ========= TODO : END ========= ###
338
339
340 class LayerNorm(nn.Module):
341     """
342     LayerNorm module as in the paper https://arxiv.org/abs/1607.06450
343
344     Note : Variance computation is done with biased variance.
345
346     Hint :
347     - You can use torch.var and specify whether to use biased variance or not.
348     """
349
350     def __init__(self, normalized_shape, eps=1e-05, elementwise_affine=True) ->
    None:
351         super().__init__()
352
353         self.normalized_shape = (normalized_shape,)
```

```
        self.eps = eps
        self.elementwise_affine = elementwise_affine

        if elementwise_affine:
            self.gamma = nn.Parameter(torch.ones(tuple(self.normalized_shape)))
            self.beta = nn.Parameter(torch.zeros(tuple(self.normalized_shape)))

    def forward(self, input):
        """
        Forward pass of the LayerNorm Layer.

        Args:
        input : torch.Tensor
            A tensor of shape (batch_size, num_tokens, token_dim) containing the input tokens.

        Output:
        torch.Tensor
            A tensor of shape (batch_size, num_tokens, token_dim) containing the output tokens.
        """

        mean = None
        var = None
        # ========= TODO : START ========= #

        mean = input.mean(dim=-1, keepdim=True)
        var = input.var(dim=-1, keepdim=True, unbiased=False)

        # ========= TODO : END ========= #

        if self.elementwise_affine:
            return (
                self.gamma * (input - mean) / torch.sqrt((var + self.eps)) + self.beta
            )
        else:
            return (input - mean) / torch.sqrt((var + self.eps))


class TransformerLayer(nn.Module):
    """
    Class definition for a single transformer layer.
    """

    def __init__(self, input_dim, num_heads, feedforward_dim=None):
        super().__init__()
        """
        Initialize the Transformer Layer.
        We will use prenorm layer where we normalize the input before applying the attention and feedforward layers.

        The model should have the following layers:
        1. A LayerNorm layer. (self.norm1)
```

```
        2. A MultiHeadAttention layer. (self.attention)
        3. A LayerNorm layer. (self.norm2)
        4. A FeedForwardLayer layer. (self.feedforward)

        NOTE : PLEASE KEEP OF EACH LAYER AS PROVIDED BELOW TO FACILITATE TESTING.
        """

        # ========= TODO : START ========= #

        self.norm1 = LayerNorm(input_dim)
        self.attention = MultiHeadAttention(input_dim, num_heads)
        self.norm2 = LayerNorm(input_dim)
        self.feedforward = FeedForwardLayer(input_dim, feedforward_dim)

        # ========= TODO : END ========= #

    def forward(self, x):
        """
        Forward pass of the Transformer Layer.

        Args:
        x : torch.Tensor
            A tensor of shape (batch_size, num_tokens, token_dim) containing the input tokens.

        Output:
        torch.Tensor
            A tensor of shape (batch_size, num_tokens, token_dim) containing the output tokens.
        """

        # ========= TODO : START ========= #

        x = x + self.attention(self.norm1(x))
        x = x + self.feedforward(self.norm2(x))
        return x

        # ========= TODO : END ========= #


class MiniGPT(nn.Module):
    """
    Putting it all together: GPT model
    """

    def __init__(self, config) -> None:
        super().__init__()
        """
        Putting it all together: our own GPT model!

        Initialize the MiniGPT model.

        The model should have the following layers:
```

```
        1. An embedding layer that maps tokens to embeddings. (self.vocab_embedding)
        2. A positional embedding layer. (self.positional_embedding) We will use learnt positional embeddings.
        3. A dropout layer for embeddings. (self.embed_dropout)
        4. Multiple TransformerLayer layers. (self.transformer_layers)
        5. A LayerNorm layer before the final layer. (self.prehead_norm)
        6. Final language Modelling head layer. (self.head) We will use weight tying (https://paperswithcode.com/method/weight-tying) and set the weights of the head layer to be the same as the vocab_embedding layer.

        NOTE: You do not need to modify anything here.
        """

        self.config = config
        self.vocab_embedding = nn.Embedding(config.vocab_size, config.embed_dim)
        self.positional_embedding = nn.Embedding(
            config.context_length, config.embed_dim
        )
        self.embed_dropout = nn.Dropout(config.embed_dropout)

        self.transformer_layers = nn.ModuleList(
            [
                TransformerLayer(
                    config.embed_dim, config.num_heads, config.feedforward_size
                )
                for _ in range(config.num_layers)
            ]
        )

        # prehead layer norm
        self.prehead_norm = LayerNorm(config.embed_dim)

        self.head = nn.Linear(
            config.embed_dim, config.vocab_size
        )  # Language modelling head

        if config.weight_tie:
            self.head.weight = self.vocab_embedding.weight

        # precreate positional indices for the positional embedding
        pos = torch.arange(0, config.context_length, dtype=torch.long)
        self.register_buffer("pos", pos, persistent=False)

        # Needed by _init_weights for GPT-2 style FFN init scaling
        self.num_layers = config.num_layers

        self.apply(self._init_weights)

    def forward(self, x):
        """
        Forward pass of the MiniGPT model.

        Remember to add the positional embeddings to your input token!!
```

```
505
506         Args:
507         x : torch.Tensor
508             A tensor of shape (batch_size, seq_len) containing the input tokens.
509
510         Output:
511         torch.Tensor
512             A tensor of shape (batch_size, seq_len, vocab_size) containing the
    logits.
513
514         Hint:
515         - You may need to 'trim' the positional embedding to match the input
    sequence length
516         """
517
518         ### ========= TODO : START ========= ###
519
520         B, T = x.shape
521         tok_emb = self.vocab_embedding(x)
522         pos_emb = self.positional_embedding(self.pos[:T])
523         h = tok_emb + pos_emb
524         h = self.embed_dropout(h)
525         for layer in self.transformer_layers:
526             h = layer(h)
527         h = self.prehead_norm(h)
528         logits = self.head(h)
529         return logits
530
531         ### ========= TODO : END ========= ###
532
533     def _init_weights(self, module):
534         """
535         Weight initialization for better convergence.
536
537         NOTE : You do not need to modify this function.
538         """
539
540         if isinstance(module, nn.Linear):
541             if module._get_name() == "fc2":
542                 # GPT-2 style FFN init
543                 torch.nn.init.normal_(
544                     module.weight, mean=0.0, std=0.02 / math.sqrt(2 *
    self.num_layers)
545                 )
546             else:
547                 torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)
548             if module.bias is not None:
549                 torch.nn.init.zeros_(module.bias)
550         elif isinstance(module, nn.Embedding):
551             torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)
552
553     def generate(self, context, max_new_tokens=100):
554         """
555         Use the model to generate new tokens given a context.
```

```
        Args:
        context : torch.Tensor
            The context is a sequence of tokens, where each token is an index in the vocabulary.
        max_new_tokens : int
            The maximum number of new tokens to generate.

        Output:
        torch.Tensor
            A tensor containing the generated tokens.

        Hint:
        - This should be similar to the Bigram Language Model, but you will use the entire context to predict the next token.
          Instead of sampling from the distribution p(x_t | x_{t-1}),
            you will sample from the distribution p(x_t | x_{t-1}, x_{t-2}, ..., x_{t-n}),
            where n is the context length.
        - When decoding for the next token, you should use the logits of the last token in the input sequence.

        """

        ### ========= TODO : START ========= ###

        self.eval()
        device = context.device
        if context.dim() == 1:
            generated = context.unsqueeze(0)
        else:
            generated = context
        with torch.no_grad():
            for _ in range(max_new_tokens):
                x_cond = generated[:, -self.config.context_length:]
                logits = self.forward(x_cond)
                logits = logits[:, -1, :]
                probs = torch.softmax(logits, dim=-1)
                next_token = torch.multinomial(probs, num_samples=1)
                generated = torch.cat([generated, next_token], dim=1)
        return generated

        ### ========= TODO : END ========= ###

    def generate_beam(self, context, max_new_tokens=100, beam_width=5, length_penalty=1.0):
        """
        Beam search decoder (Bonus).

        Args:
            context : 1D tensor (T,) or 2D (1, T) of token ids.
            max_new_tokens : how many tokens to generate.
            beam_width : number of beams k.
            length_penalty : alpha used in score / length^alpha.
```

```
        Returns:
            best_seq : 1D tensor of the highest-scoring sequence.
        """
        self.eval()
        device = context.device
        if context.dim() == 1:
            context = context.unsqueeze(0)

        beams = [(context, 0.0)]

        with torch.no_grad():
            for _ in range(max_new_tokens):
                candidates = []
                for seq, score in beams:
                    x_cond = seq[:, -self.config.context_length:]
                    logits = self.forward(x_cond)
                    logits = logits[:, -1, :]
                    log_probs = torch.log_softmax(logits, dim=-1)

                    topk_logp, topk_idx = log_probs.topk(beam_width, dim=-1)
                    for b in range(beam_width):
                        next_tok = topk_idx[0, b].view(1, 1)
                        next_logp = topk_logp[0, b].item()
                        new_seq = torch.cat([seq, next_tok], dim=1)
                        new_score = score + next_logp
                        candidates.append((new_seq, new_score))

                candidates.sort(
                    key=lambda x: x[1] / ((x[0].shape[1]) ** length_penalty),
                    reverse=True,
                )
                beams = candidates[:beam_width]

        beams.sort(
            key=lambda x: x[1] / ((x[0].shape[1]) ** length_penalty),
            reverse=True,
        )
        best_seq, _ = beams[0]
        return best_seq.squeeze(0)
```